Closure

→ retains the access to its outer function's variable, even after the outer function has finished executing.

→ It "remembers" the enviroment in which it was created.

```
function outer() {
  let outerVar = "Outer";
  function inner() {
    outerVar = "Update";
  }
  return inner;
}

const closure = outer();
closure(); // Outer
closure(); // Updated
```

Key concepts

1. Lexical Scoping

Closure rely on lexical scope → a function's scope is determind by where it is defined, not where it is executed.

2. Private Variables

Closures allow hiding variables from outside word, giving encapsulation.

When & Why to use :

- To create private varibles
- To maintain state.
- To work with async code
- To implement → module pattern

1. Private Variable

```
function counter() {
    let count = 0;
    return function() {
        count++;
        return count;
    };
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
console.log(increment()); // 3
```

2. Module Pattern

```
const counter = (function() {
    let count = 0;
```

```
    return {
        increment: function () {
            console.log(++count);},
        reset: function () {
            count = 0; console.log("Reset");}
    };
})();

counter.increment();
counter.increment();
counter.reset();
```

3. Closures with setTimout

```
function createTimers() {
    for(let i = 1; i <= 3; i++) {
        setTimeout(() => {
            console.log(`Timer ${i}`);
        }, i * 1000);
    }
}
createTimers();
```

4. Closure with this

```
function Person(name){
    this.name = name;
    setTimeout(function(){
        console.log(this.name);
    }.bind(this), 1000);
}

const G1 = new Person("XYZ");
```

5. Function Currying

```
function add(a){
    return function(b){
        return a+b;
    };
}

const addTwo = add(2);
```

```
console.log(addTwo(3)); // 5
console.log(addTwo(4)); // 6
```

6. Real-world closure event listener

```
<button id="clickBtn">Click Me</button>
<script>
  function createClickCounter() {
      let count = 0;
      return function() {
          count++;
          console.log(count);
      };
  }
  const button = document.getElementById
  ("clickBTn");
  button.addEventListner("click",
                createClickCounte());
</script>
```

How does scope chaining work

The scope chain in Javascript is the hierarchical order in which the Javascript engines looks for variable and functions when they are refered.

→ **Lexical scoping :** The scope chain is determined by physical placement of code during writing.

→ **Lookup Process :**

When a variable or function is accessed, the engine first seaches in current scope.

→ **Traversing up :** If not found in current scope engine move to upper scope (parent scope).

→ Global Scope: The process continue until global scope is reached.

→ Reference Error: if variable is not found anywher ReferenceErro is thrown.

Memory Leak in closure

→ if closure holds references to large objects

→ a closure keep reference to variable in its outer scope, even if those variable are no longer needed.

→ If the closure is long-lived like an even listner) then variable it closes over also stay in memory.